

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	M. Deepilca	Reg. No.:	192372296
Section / Batch:	2023	Date:	04/09/2026

Code of Conduct

I M. Deepilca / 192372296 (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: dyf. M.

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

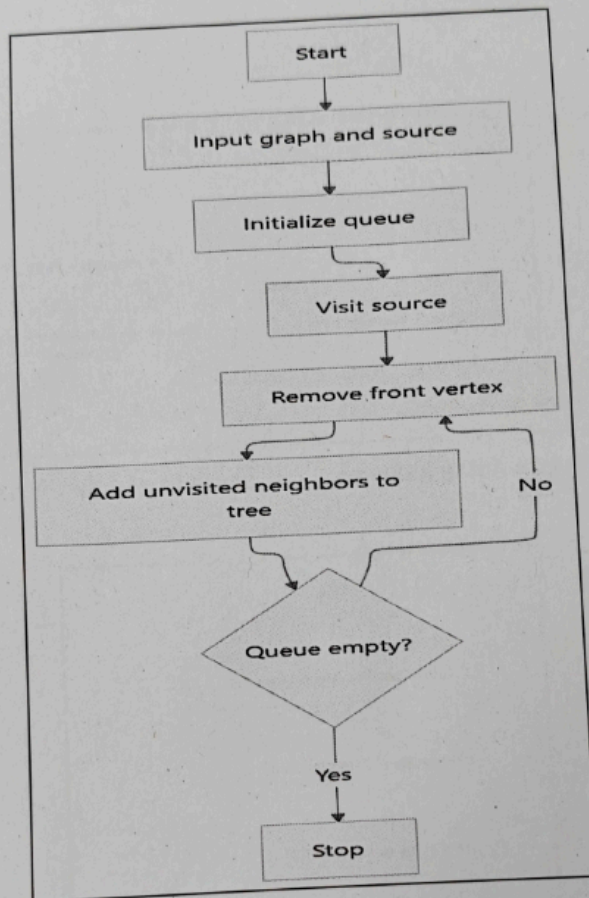
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A – Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

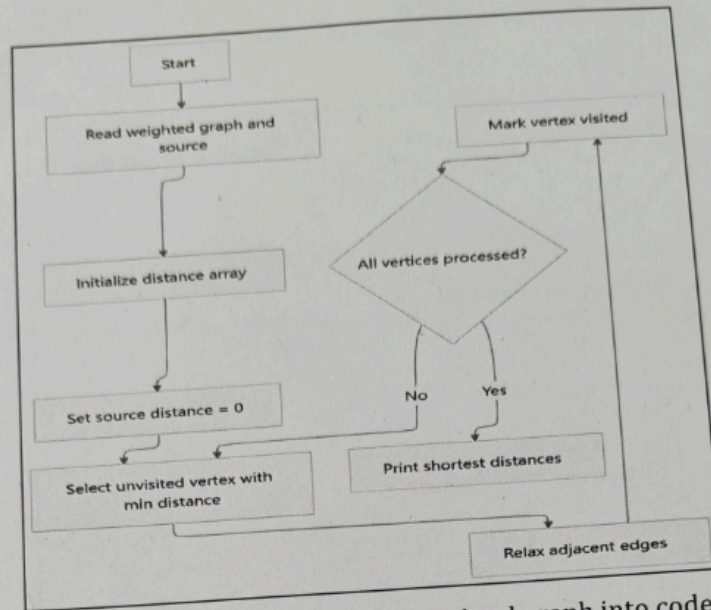
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

(20 Marks)

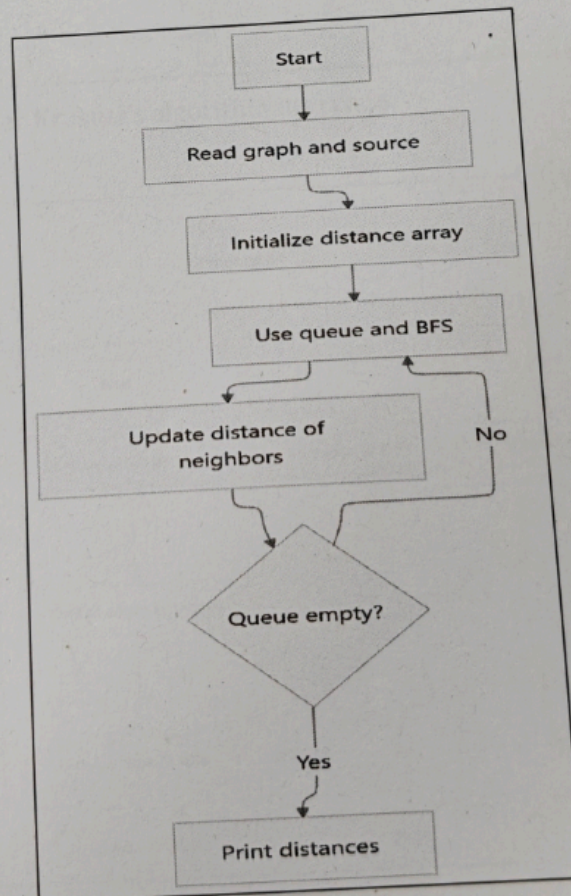
Flowchart Diagram:



3. Convert the flowchart for shortest path in an unweighted graph into code.

(20 Marks)

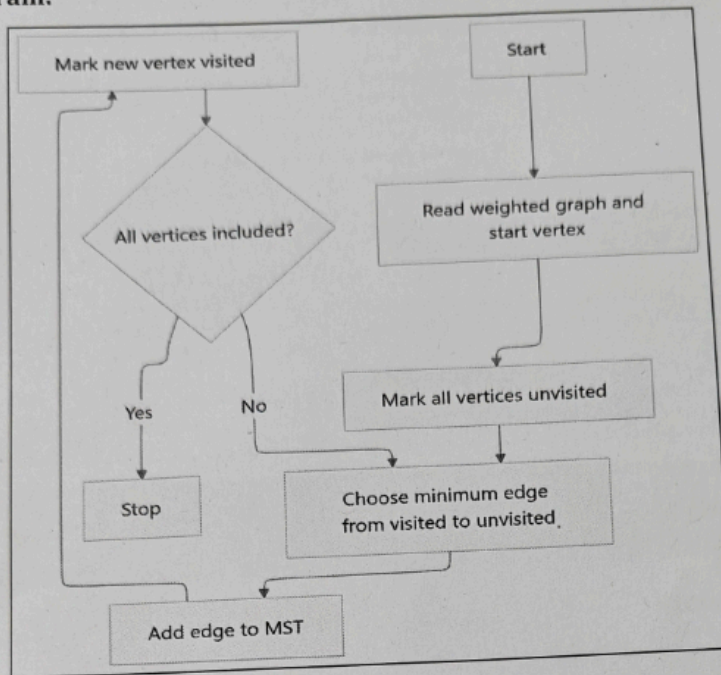
Flowchart Diagram:



4. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

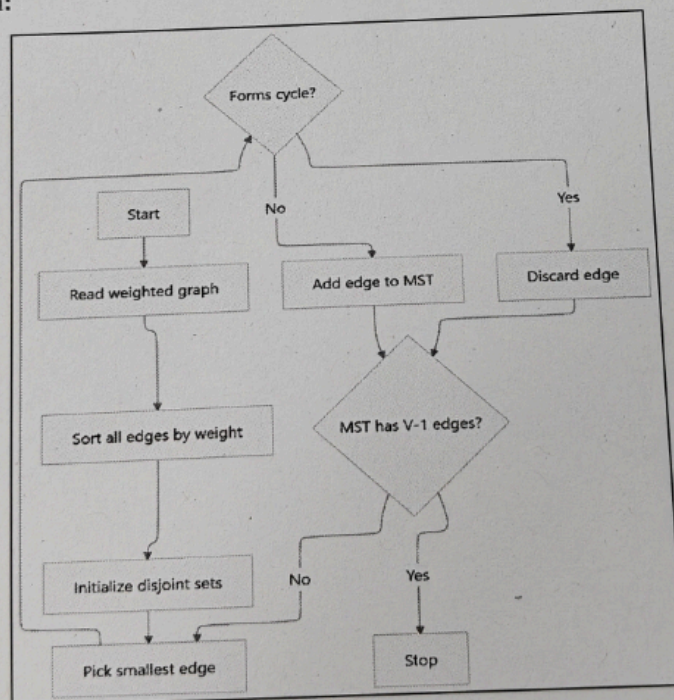
Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code.

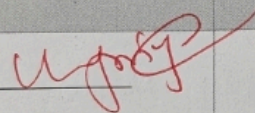
(20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

convert the flowchart for generating a bfs tree into code.

from collections import deque

def bfs-tree (graph, source):

visited = set ([source]).

queue = deque ([source])

bfs-tree-edges = []

traversal-ords = []

while queue:

u = queue.popleft()

traversal-ords.append(u)

for v in graph.get(u, []):

if v not in visited:

visited.add(v)

queue.append(v)

bfs-tree-edges.append((u,v)).

return traversal-ords-bfs-tree-edges.

2) Convert the flowchart for Dijkstra's shortest path algorithm into code:

import heapq

def dijkstra (graph, source):

dist = {node: float('inf') for node in graph}.

```
dist[source] = 0  
visited = set()  
pq = [(0, source)]
```

```
while pq:
```

```
    d, u = heapq.heappop(pq)
```

```
    if u in visited:
```

```
        continue
```

```
    visited.add(u)
```

```
    for v, w in graph.get(u, []):
```

```
        if dist[v] > d + w:
```

```
            dist[v] = d + w
```

```
            heapq.heappush(pq, (dist[v], v))
```

```
return dist
```

3) Convert the flowchart for shortest path in an unweighted graph into code.

```
from collections import deque
```

```
def shortest_path_unweighted(graph, source):
```

```
    dist = {node: -1 for node in graph}
```

```
    dist[source] = 0
```

```
    queue = deque([source])
```

```
    while queue:
```

```
        u = queue.popleft()
```

if v in graph, get $(u, [])$:

if $\text{dist}[v] = -1$:

$\text{dist}[v] = \text{dist}[u] + 1$

queue.append(v).

return dist

convert the flowchart for prisms algorithm into code.

import heapq

def prisms_mst(graph, start):

visited = set([start])

mst = []

: pq = []

for u, w in graph.get(start, []):

heapq.heappush(pq, (w, start, v))

total_cost = 0

while pq and len(visited) < len(graph):

w, u, v = heapq.heappop(pq).

if u in visited:

heapq.heappush(pq, (w, u , v , next)).

return mst, total_cost.

5)

Convert the flowchart for Kruskal's algorithm into code.

```
class DSU:
```

```
    def __init__(self, n):
```

```
        self.parent = list(range(n))
```

```
    def find(self, x):
```

```
        if self.parent[x] != x:
```

```
            self.parent[x] = self.find(self.parent[x])
```

```
        return self.parent[x]
```

```
    def union(self, x, y):
```

```
        rx, ry = self.find(x), self.find(y)
```

```
        if rx == ry:
```

```
            return False
```

```
            self.parent[ry] = rx
```

```
            return True
```

```
def kruskal_mst(vertices, edges):
```

```
    edges.sort(key = lambda x: x[2])
```

```
    dsu = DSU(vertices)
```

```
    mst = []
```

```
    total_cost = 0
```

```
    for u, v, w in edges:
```

```
        if dsu.union(u, v):
```

algorithm n,

mst.append((u, v, w))

total_cost += w

if len(mst) == vertices - 1:

break

return mst, total_cost,

Ujjwal